Redesign Mixture-of-Experts Routers with Manifold Power Iteration

Songhao Wu · Ang Lv · Ruobing Xie · Yankai Lin

ABSTRACT

Router is the cornerstone component to the Mixture-of-Experts models. Serving as expert proxies, the rows of the router matrix compute their similarity to the MoE inputs to determine which subset of experts is activated. Ideally, each router row is designed to encode the expert matrix into this representative vector, such that its dot-product with token can better reflect token-expert affinity. However, there exists no design principles to enforce this condensation. In this paper, we propose to align each router row with the principal singular direction of the associated expert, as this direction provides the most expressive mathematical description of a matrix. Based on this principle, we propose a router redesign with Manifold Power Iteration (MPI). Specifically, it introduces a "Power-then-Retract" paradigm, where a power iteration step is performed on the router weights, followed by a retraction to impose a norm constraint to ensure both efficiency and stability. Theoretically, we show that MPI drives router rows to converge toward the principal singular directions of associated experts. Empirically, we pretrain MoE model across scales from 1B to 11B parameters to confirm that this alignment facilitates more effective MoE models.

1 Introduction

Mixture-of-Experts (MoE, 6, 10, 23, 25) stands as a pivotal model architecture in LLMs to scale model capacity with a constrained computational budget. Specifically, it replaces standard Transformer Feed Forward Networks (FFNs) with an ensemble of expert modules, using a router to select experts per token for sparse activation. MoE enables more efficient training given the same computation budget, paving the way for LLM training with trillions of parameters [6, 39].

At the heart of MoE lies the router, which is typically parameterized as a linear matrix. For each input token, the router computes similarity scores against the matrix rows and dispatches the token to the experts corresponding to the top-scoring rows. While this design is straightforward and has long been accepted as a matter of course, we challenge this conventional wisdom. Ideally, each individual row in MoE router matrix should faithfully reflect the expert's intrinsic features. The router matrix can thus better ground the identity

of each expert, allowing token–router affinity to serve as a precise proxy for token–expert assignment. However, there lacks a constraint in MoE router to enforce the encoding of expert features into router rows of limited expressivity. This absence may lead to suboptimal router design, compromising both training convergence and competence of MoE models.

We propose to align each router row with the principal singular direction of its corresponding expert's weight matrix. This choice is grounded in a linear algebraic intuition: the principal singular direction preserves the highest density of information within a matrix [11, 13], making it the optimal compressed representation to characterize that matrix. Since each expert module is parameterized as weight matrices, encoding it into a single router vector is exactly the task of capturing its most informative direction. To avoid the prohibitive cost of exact singular value decomposition (SVD), we leverage power iteration [13] as a lightweight alternative to obtain this principal direction online. Specifically, the power iteration scheme uses only standard matrix-vector products to solve for the principal singular vector, obviating the need for expensive full matrix factorization.

In practice, we perform only one single power iteration on the router weights during each training step. After that, a retraction step is introduced to regularize the L_2 norm of the router weights, maintaining them at a constant scale to prevent potential explosion or collapse. This "Power-then-Retract" paradigm gives our design its name: Routers with Manifold Power-Iteration (MPI). We prove that this online update rule is equivalent to a steepest ascent optimization that maximize the router's projection onto the expert weight under the principle of minimal updates. From a theoretical perspective, this confirms that each update step drives an adaptive convergence of router rows toward the principal singular direction of their associated experts. Consequently, this imposes an explicit constraint on router optimization to encode the most dominant expert features into router vectors, which has been overlooked in conventional router designs.

We conduct extensive pretraining experiments across a wide range of MoE model scales using billions of tokens. We contend that our router redesign with Manifold Power Iteration presents a fundamental departure from conventional MoE routers and brings intrinsic improvements. While it retains the standard interface of MoE routers, it provides a fresh perspective to rethink the interplay between routers and experts. Empirical evaluations, scaling up to 11B parameters, show that MoE with MPI consistently facilitates faster convergence, superior downstream performance, and improved load balancing. We further demonstrate that this superiority is robust to shifts in model features, stemming entirely from our principled router design. We hope these insights shed light on the intrinsic nature of MoE router and inspire future exploration.

2 Background: Mixture-of-Experts

We center our discussion on MoE-based LLMs. The key component of an MoE is the router, which dispatches inputs to a sparse subset of the experts. Typically, the router employs a 2D linear weight matrix $R \in \mathbb{R}^{N \times D}$ to project the input $x \in \mathbb{R}^{D}$ into gating weight vector w

over the N experts:

$$w = Softmax\left(TopK\left(xR^{\top}\right)\right). \tag{1}$$

where experts with the top-K largest gating weights are selected. MoE layer output is then computed as weighted sum of the selected experts:

$$\mathrm{MoE}\,(x) \;=\; \sum_{k=1}^{K} w_k \cdot E_k(x), \tag{2}$$

where each expert module is Gated Linear Unit [36] with the Swish [32] activation function:

$$E_k(x) = \left(SiLU(xW_g^k) \odot (xW_p^k)\right) W_o^k \,.$$

While this router design is straightforward and sufficient in most cases, certain limitations persist at the design stage and hinder its optimal performance. For instance, no explicit constraint is imposed to ensure that routers can faithfully reflect the experts' intrinsic features. For input x, its affinity with the i-th expert is defined as its inner product with $R_{[i]}$. Ideally, $R_{[i]}$ should maximally preserve the geometry of the i-th expert weights to better act as a feature vector; however, such a constraint is absent and may result in suboptimal convergence as a consequence. Leveraging this insight, we propose a redesign of MoE router and empirically confirm its effectiveness in the following sections.

3 Methodology

We first elucidate our motivation, derive framework for Manifold Power Iteration and interpret some key design principles. We then revisit the essence of our method from optimization perspective and provide accessible insights into how it works.

3.1 Motivation

In MoE routing, $\mathrm{R}_{[i]}$ is designed to serve as a representative vector for the i-th expert, ensuring that its inner product with an input faithfully reflects their mutual affinity. Consequently, the token is routed to the experts with the highest affinity scores. This suggests that an ideal $R_{[i]}$ should be optimized to effectively encode the distinctive characteristics of the expert matrix W_*^i within a constrained vector space. From a matrix-theoretic perspective, a vector is best aligned with a matrix's principal singular directions to capture its most essential traits [8]. In the context of MoE, this principle dictates that a well-coupled router $R_{[i]}$ should be guided toward the principal singular direction of expert weights W_*^i. Geometrically, this is equivalent to maximizing squared projection of $\mathrm{R}_{[i]}$ onto the row space spanned by W_*^i, which is given by:

$$\max_{R_{[i]}} \quad \phi 0(W_*^i,\, R_{[i]}) = \frac{\|R_{[i]} W_*^i\|_2^2}{\|R_{[i]}\|_2^2} \tag{3}$$

where $\phi 0(\cdot)$ is the objective function, also known as the Rayleigh quotient with $W_*^i W_*^{i\top}$ and $R_{[i]}$[1]

[1] Unless specified otherwise, we substitute W_g^i for $\mathrm{W}_*^i \in \{\mathrm{W}_g^i, \mathrm{W}_o^i, \mathrm{W}_p^i\}$ hereafter for the sake of simplicity.

However, it is prohibitive to execute an exact singular value decomposition (SVD) for all expert matrices to obtain the principal singular vector at each training step. To address the issue, we leverage power iteration onto $R_{[i]}$ as a lightweight alternative to SVD. Backed by power method theory [11], it enables $\mathrm{R}_{[i]}$ to progressively track and converge toward the principal singular direction over training steps through efficient matrix-vector products. This motivates the core design of our routers, which is built upon Power Iteration followed by a row-wise normalization. We first outline the implementation details and defer an in-depth discussion to a later section.

3.2 Routers with Manifold Power-Iteration

Manifold Power-Iteration.Specifically, the proposed approach follows a "Power-then-Retract" paradigm, which involves (1) a power iteration step that aligns the router with the principal direction of expert weights, followed by (2) a L_2 retraction step for weight containment and numerical stability.

For an arbitrary row $R_{[i]}$ of the router weights, we first fetch its associated expert weights W_g^i and perform a single step of power iteration on it:

$$\hat{R_{[i]}} \;=\; R_{[i]}\, W_g^i\, W_g^{i\top}. \tag{4}$$

The cumulative execution of power iteration across training steps can induce numerical instability, causing L_2 norm of $\hat{\mathrm{R}}_{[i]}$ to diverge. To counteract this divergence, we constrain the L_2 norm of $\hat{\mathrm{R}}_{[i]}$ to a hyperparameter C after each iteration:

$$R'_{[i]} = C \cdot \frac{\hat{R_{[i]}}}{\|\hat{R_{[i]}}\|_2}, \tag{5}$$

while designed to avoid instability, this retraction provides additional benefits. Conceptually, it rectifies the potential expert bias induced by scale disparities in router norms, where an amplified norm can easily inflate gating weights and consequently overload the corresponding expert. Based on these two designs, the original router matrix R is substituted with the concatenated formulation:

$$R'^{\top} = \left[\, \hat{R_{[1]}} \mid \hat{R_{[2]}} \mid \cdots \mid \hat{R_{[N]}} \,\right],$$

and the final gating weights w are recomputed as:

$$w' = Softmax\left(TopK\left(xR'^{\top}\right)\right). \tag{6}$$

We designate this refined router R' as Routers with Manifold Power-Iteration (MPI), to fully manifest the Power-then-Retract paradigm.

Figure 1 provides a Pytorch-style pseudo-code to help understand our implementation.

Design Principle.We also establish a principle to guide the configuration of C. To this end, we introduce an assumption that routing logits should be bounded at a constant scale to to prevent explosion, inspired by insights in [29]:

$$\| xR'^{\top} \|_\infty \;\sim\; O(1),$$

```python
from torch.nn.functional import normalize
from megablocks.layers.moe import MoE
class MoE_MPI(MoE):
    def foward(self, x, C_prime = 1):
        # R: [N, D], wg: [N, D, d]
        # PowerIter: ([N, 1, D] @ [N, D, d])
        #   @ [N, d, D] -> [N, D]
        (*\bluebg*) R_hat = (self.R.unsqueeze(1) @ wg.
        (*\bluebg*)    transpose(1, 2) @ wg).squeeze()
        # Retracition: [N, D] -> [N, D]
        (*\pinkbg*) R_prime = normalize(R_hat, p=2, dim=-1)
        (*\pinkbg*) C = C_prime * (N ** -0.5)
        (*\pinkbg*) logits = C * (x @ R_prime.T)
        s_prime = logits.softmax(dim=-1)
        w_prime, _ = torch.topk(s_prime, dim=-1)
        return self.experts(x, s_prime, w_prime)
```

FIGURE 1 Pseudo code for Manifold Power-Iteration.

Given a scale-invariant x, this upper bound inherently implies that $C \sim \Theta(\frac{1}{\sqrt{N}})$ with respect to N. This is evidenced by the following derivation:

$$\| xR'^{\top} \|_\infty \leq \sqrt{\sum_{i=1}^{N} (xR_{[i]}^{\top})^2} \sim O(C\sqrt{N}),$$
$$\textit{where} \quad xR_{[i]}^{\top} \sim O(C) \quad \text{for each expert.} \tag{7}$$

Therefore, to enforce the $O(1)$ ceiling and decouple the scaling effect from the expert count N, we introduce a redefinition: $C \coloneqq \frac{C'}{\sqrt{N}}$, where C' is a scale-invariant global hyperparameter.

3.3 From Maximum Projection Constraints to Manifold Power-Iteration

Section 3.1 provides an intuitive motivation explaining the introduction of Power-Iteration into our router design. This section extends the maximum projection objective in Eq. 3 to align with our formulation, which can be expressed as:

$$\max_{\Delta r} \quad \Phi 0(W_g, R'_{[i]} + \Delta r) \tag{8}$$
$$\text{s.t. } \|R'_{[i]}\|_2 = \|R'_{[i]} + \Delta r\|_2 = C, \quad \|\Delta r\|_2 \leq \eta,$$

since the normalization ensures a constant denominator, the original objective reduces to:

$$\Phi 0(W_*^i, R'_{[i]}) = \|R'_{[i]} W_*^i\|_2^2 = R'_{[i]} W_*^i W_*^{i\top} R'^{\top}_{[i]},$$

Δr represents the update, and η constrains the update within a small bounded region. We impose norm constraints on both $R'_{[i]}$ and $R'_{[i]} + \Delta r$. To analyze this optimization landscape,

we consider a first-order Taylor approximation of the objective:

$$\Phi 0(W_g,\, R'_{[i]} + \Delta r) \;=\; \Phi 0(W_g,\, R'_{[i]}) \;+\; \langle G,\, \Delta r \rangle,$$

where $G = 2\, R'_{[i]} W_g W_g^{\top}$ represents the gradient of $R_{[i]}$. The Taylor approximation reduces the objective to maximizing the inner product $\langle G,\, \Delta r \rangle$. To satisfy the norm constraint and ensure that the updated router $R'_{[i]}$ remains on the spherical manifold, we project the gradient G onto the tangent space of the sphere. Defining $M \coloneqq W_g W_g^{\top}$ to simplify notation, and setting $C = 1$ without loss of generality, the gradient ascent update Δr_g on the manifold is formulated as:

$$\begin{aligned} \Delta r_g &= \eta\; G \left(I - \frac{R_{[i]}'^{\top} R'_{[i]}}{R'_{[i]} R_{[i]}'^{\top}} \right) \\ &= \eta \left(R'_{[i]} M - R'_{[i]} \left(R'_{[i]} M R_{[i]}'^{\top} \right) \right), \end{aligned} \tag{9}$$

where the scaling constants are absorbed into η. We also derive an approximation for the exact update Δr_M introduced by Manifold Power-Iteration:

$$\Delta r_M \approx \frac{1}{R'_{[i]} M R_{[i]}'^{\top}} \left(R'_{[i]} M - R'_{[i]} (R'_{[i]} M R_{[i]}'^{\top}) \right). \tag{10}$$

By comparing steepest ascent (Δr_g) with our router update (Δr_M), we observe a striking structural alignment. In this light, our proposed router design constitutes an optimization tailored for maximum projection constraints with an adaptive step-size. Specifically, it drives a steady convergence of the router weights toward the principal singular vector, with the step size decreasing and updates becoming more careful as $R'_{[i]}$ are mostly aligned with the principal direction of W_g^i (i.e. Δr_M is moderated because the denominator $R'_{[i]} M R_{[i]}'^{\top}$ in Eq. 10 increases), and vice versa.

The update can also be interpreted from an SVD perspective. After sufficiently many training steps (or power iterations), the term $R'_{[i]} M$ in Eq. 10 approaches the dominant singular vector of W_g. At that stage, the scalar quantity $R'_{[i]} M R_{[i]}'^{\top}$ corresponds to the L_2 norm when feeding $R'_{[i]}$ into W_g, yielding a scalar that scales $R'_{[i]}$. The subtraction term in Eq. 10 therefore derives an update direction that points toward the residual mismatch between $R_{[i]}$ and the dominant singular vector. Applying the update progressively rotates $R_{[i]}$ toward the principal singular subspace of W_g.

These interpretations help explain why the proposed method effectively optimizes the router to encode the most informative expert features. We provide supplementary derivation in Appendix A and hope this perspective can inspire readers.

4 Experiment

4.1 MPI is an Optimizer-Agnostic Design

We first pretrain 1B MoE models using different optimizers, guided by two primary motivations:

(1) To substantiate that MPI is an intrinsic improvement to router design, which remains agnostic to shifts in model features across optimizers

(2) To provide a foundational analysis to justify our setup for the large-scale experiments.

Specifically, we pretrain these 1B models with AdamW [21] and Muon [16], alongside their Hyperball Opimization [42] variants, AdamH and MuonH.[2] Detailed model and optimizer configurations are provided in Appendix B.

We pretrain the baselines on 100B tokens and analyze the resulting convergence and downstream performance. Figure 2 plots the convergence comparisons, using MuonH as representative. Table 1 reports the average downstream performance over a task suite of 25 benchmarks. Crucially, MoE with MPI achieves both accelerated convergence and improved downstream performance across all optimizer setups at the 1B scale. Motivated by this, we scale our experiments to confirm these benefits at larger capacities. We select MuonH for the large-scale experiments owing to (1) its hyperparameter transferability, and (2) its optimal convergence performance among all 1B MoE baselines.

4.2 Comparative Analysis with vanilla MoE

We pretrain MoE with MPI at two larger scales: 3B and 11B. All models are pretrained on 350B tokens sampled from FineWeb-Edu dataset [22], with 1B tokens reserved to serve as the validation set. We then midtrain the models on 100B tokens from 24. Full architecture hyperparameters, training configurations are available in Appendix B. We forgo comparisons with other baselines since our design conforms to the standard router form and is theoretically orthogonal to these studies. Section 6 explores this compatibility to provide an investigation.

PPL (↓)	Validation	Math	Code
MoE 3B	0.764	1.688	1.376
MoE *w.* MPI	0.754	1.581	1.296
MoE 11B	0.728	1.852	1.263
MoE *w.* MPI	0.723	1.581	1.259

TABLE 2 Perplexity in bits per byte for MoE with MPI.

Convergence and Performance.We conduct pretraining on two scales and confirm that MoE with MPI achieves faster convergence and improved downstream performance. Figure 3 (a) and (b) present a comparison of pretraining loss and downstream performance evolution for 11B MoE.

We observe that MoE with MPI leads to more effective training and maintains this loss advantage throughout. We also report perplexity comparison evaluated on both validation set and held-out Math and Code sets from Olmo 3 [24]. As shown in Table 2, the advantage

[2] For readers unfamiliar with these advanced optimizers, please refer to the related work section (Section 7).

Task (→)	ARC-C	MMLU	TriviaQA	NaturalQs	BBH	GSM8K	MBPP	AVG.
Setup (→)	5-shot	5-shot	5-shot	5-shot	3-shot CoT	8-shot CoT	3-shot	
MoE 3B	55.91	47.01	45.78	17.87	29.53	16.22	42.25	36.37
w. MPI	58.96	48.83	46.52	20.13	30.99	20.92	44.54	38.70
MoE 11B	61.54	50.00	55.41	25.30	31.17	17.89	45.12	40.92
w. MPI	62.24	50.93	56.89	25.36	31.45	27.60	44.87	42.76

TABLE 3 Performance of MoE with Manifold Power Iteration on challenging benchmarks at both 3B and 11B scales.

for language modeling remains consistent across all domains.

We evaluate downstream tasks to confirm that the loss reduction manifests as superior model competence. Specifically, we use a suite of 9 core tasks to monitor and Figure 3 (b) plots the evolution of average accuracy throughout pretraining. We observe that MPI maintains the advantage on downstream tasks throughout pretraining. Furthermore, we extend our evaluation to more challenge tasks after mid-training, including benchmarks across knowledge-intensive QA [3, 14], reading comprehension [17, 18], language understanding and reasoning [37], math skills and code generation [1, 4]. As summarized in Table 3, MoE with MPI delivers consistent performance gain, which further validates the effectiveness of our router design. Our evaluation setups are available in Appendix C.

Load Balancing.As shown in Figure 4, a noticeable decrease in balance loss is observed for MoE with MPI during pretraining. Specifically, this loss drops sharply during the early stages and remains at a low level thereafter. We suspect that this reduction might be an artifact of router retraction. Therefore, we report MaxVio on validation set as a more accurate reflection of load balance.

Table 4 reports both MaxVio_{Batch} and MaxVio_{Global} of different models. The reported MaxVio confirms that MPI is compatible with the load balancing loss and achieves a more equitable load distribution than the vanilla MoE as an unexpected bonus. Tentatively, we attribute the improved balance to our retraction design, and leave a deeper investigation into it to future work.

Efficiency Analysis.We provide a breakdown efficiency analysis into MoE with MPI to confirm its practicality in large-scale MoE pretraining.

(1) Our router design introduces negligible overhead with respect to training efficiency. In our 11B pretraining experiments, vanilla MoE sustains a throughput of 34.97 billion tokens per day, while MPI incurs a mere slowdown of 0.2%. Intuitively, the computational cost exerted by MPI does not exceed that of N extra tokens, which is a negligible fraction of the total tokens per batch. Our MPI design introduces zero communication overhead and avoids conflicts with standard training frameworks.

(2) At inference time, the router weights can be pre-computed with power iteration

$\text{MaxVio}_{\text{Batch}}$ (↓)		$\text{MaxVio}_{\text{Global}}$ (↓)	
MoE	*w.* MPI	MoE	*w.* MPI
1.133	1.024	0.964	0.711

TABLE 4 MaxVio comparisons for 3B MoE with MPI.

Layer	1	2	3	4	5	6	7	8	9	10	11	12
MoE	0.37	0.31	0.31	0.28	0.28	0.25	0.24	0.23	0.24	0.24	0.22	0.26
w. MPI	0.67	0.66	0.69	0.70	0.68	0.67	0.64	0.63	0.62	0.62	0.62	0.69

TABLE 5 Comparison of λ distributions. Router with Manifold Power Iteration exhibits an enhanced alignment between $R'_{[i]}$ and the principal singular direction of expert weights, manifested by significantly larger λ values.

as the model loads. Therefore, our design incurs zero inference overhead and maintains compatible with standard inference engines out-of-the-box.

Taken together, we believe in scalability of MPI toward larger-scale MoE training and deployment.

5 Method Analysis

5.1 Enhanced Router-Expert Alignment Along the Principle Singular direction

We perform a post-hoc parameter analysis to verify that our design better aligns router rows with the principal singular vector of the associated experts. Following Section 3.2, we report the projection of $R'_{[i]}$ onto W_g^i as the quantitative metric:

$$\lambda = \frac{\|R'_{[i]} W_g^i\|_2}{\|R'_{[i]}\|_2 \, \|W_g^i\|_2}, \tag{11}$$

where λ is normalized by the spectral norm to constrain within $[0, 1]$. Table 5 compares the λ distributions, where the average values across experts per layer are reported. Compared to vanilla MoE, MoE with MPI achieves a tighter couple of router vectors with the principal directions of expert weights, manifest as a prominently higher λ.

The analysis in Section 3.3 explains why a single power iteration suffices to achieve router–expert alignment along the principal singular direction. Readers may wonder

whether additional iterations could further enhance through a tighter alignment. To investigate this, we increase the iteration count to 10 to ensure full convergence. We observe that this more precise estimation results in a 5% lower throughput, and provides no further convergence advantage or downstream performance improvement (with a pre-training loss increase of 0.002 to 0.003 and a downstream drop of 1.39 percentage points). In our view, aggressive alignment disrupt the stability of router optimization, making a single power iteration a more robust and efficient choice.

5.2 Ablation Studies

We conduct ablation studies to validate the design choices of routers with Manifold Power-Iteration.

5.2.1 Impact of the Key Design Choices

We pretrain ablated 3B models on 200B tokens to validate the effectiveness of the two core designs: (1) Power Iteration and (2) Router Retraction.

Ablation on Power Iteration Design.We introduce a baseline that solely performs row-wise normalization on router weights R. This replaces the original $R'_{[i]}$ with $R^{\text{np}}_{[i]}$, which is defined as:

$$R^{np}_{[i]} = C \cdot \frac{R_{[i]}}{\|R_{[i]}\|_2}.$$

As shown in Figure 5, this ablated variant underperforms our routers with MPI, achieving nearly identical performance to the vanilla MoE. This confirms that our improvements cannot be attributed to router weights retraction. However, we observe that it exhibits a similar balance loss distribution to that of our MPI. We leave it to future work to investigate whether this normalization can lead to improved load balancing as a side benefit.

Router Retraction Enables Stable Training.To resolve the instability caused by power iteration, we adopt router retraction to mitigate the risk of L_2-Norm explosion or collapse. We replace $R'_{[i]}$ with $\hat{R_{[i]}}$ and first conduct ablation on 1B models.

Specifically, we observe loss spikes and abnormal gradients for 1B baselines pretrained with AdamW and Muon. In the absence of router retraction, the power iteration destabilizes pretraining and leads to suboptimal model convergence. While hyperball optimization can relieve this instability, it impose no constaint on the spectral norm of expert matrices, risking L_2 norm collapse as N increases. Although the ablated variant remains competitive on downstream tasks, we observe its elevation in pretraining loss of 0.003. Combining our empircial observations and analysis, we strongly advocate for this retraction design.

5.2.2 Sensitivity Analysis of Constant C

We benchmark small-scale MoE models with 256 experts, conducting a hyperparameter search over over $C' \in \{1, 2, 4, 8\}$. Each model variant is pretrained on 50B tokens and optimized with MuonH. Table 6 presents the validation perplexity (PPL) across different choices of C'.

C'	1	2	4	8	MoE
Val PPL	0.8896	0.8547	0.8533	0.8563	0.8884

TABLE 6 Validation perplexity across choices of C'.

Specifically, we have the following observations: (1) In most cases, MoE with MPI outperforms the vanilla MoE, which demonstrate that our router design is relatively insensitive to the choice of C'; (2) The optimal choice of C' basically aligns with the design principles we established in Section 3.2. Leveraging the Hyperball Optimization properties, we directly transfer the optimal C' identified in small-scale sweep into our 11B pretraining. As is shown in Section 4.2, no performance collapse is observed. We argue that this hyperparameter is mostly insensitive and transferable with the help of advanced optimizer designs.

5.3 Expert Weight Choice for Power Iteration

We pretrain 1B MoE baselines on 50B tokens to explore the optimal choice among the three candidate expert weight matrices (W_g, W_p and W_o) for power iteration. No significant divergence in pre-training loss or downstream performance is observed for these candidates. Therefore, we adopt W_g as our default choice, as it holds a marginal advantage across all candidates in current experimental setup. We leave it to future work to explore the potential of expert matrices combinations.

6 Compatibility of Manifold Power- Iteration with other Router Designs

Routers with MPI preserve the gating weights computation and modify only the router weights. Conceptually, this refinement is orthogonal to most alternative router designs. We pretrain 1B baselines on 50B tokens to explore this compatibility.

Auxiliary loss for MoE.In standard MoE practices, auxiliary losses are designed to regularize routing to address specific issues (load balance, expert specialization etc.). Section 4.2 confirms the compatibility of our router design with load balancing loss. We further integrate our method with router z-loss using a coefficient of 0.001 [45]. Our small-scale trials exhibit no loss or gradient anomalies, and the z-loss variant yields a 0.68-point improvement in downstream tasks, further confirming its compatibility.

Alternative of Activation functions.By default, we adopt Softmax as the activation function. In this section, we also explore Sigmoid as an alternative. Specifically, we fix $C = 1$ without searching to align with the Frobenius norm of MuonH. Compared with Softmax, the pretraining loss advantage narrows, while downstream performance stills improves from 41.64 to 42.05. We reserve thorough exploration on Sigmoid and other activation functions

to future work.

7 Related Work

We provide an overview of the optimizers used in this paper, which are well-established for model convergence acceleration. Beyond convergence, we seek to leverage their scalability, in the hope that our empirical insights, from model up to 11B parameters trained on 350B tokens, can be extrapolated and validated efficacy at larger scales.

We begin with an introduction of Muon [16], which orthogonalizes momentum with Newton-Schulz iterations to update parameters. Recent studies have validated its effectiveness in pretraining models with up to trillions of parameters [6, 39]. Further analysis interprets it as a steepest descent under spectral norm, which inspires other norm-constrained optimizer designs [29].

More recently, a line of work proposes imposing norm constraints on both weights and updates [42, 43]. The intuition behind is that norm constraints on weights enable stable and scalable optimization, which in turn accelerates convergence across scales and allows for hyperparameter transfer without further tuning. This paper provides a preliminary practice of these optimizers and empirically validates their effectiveness.

8 Conclusion

We revisited the design of MoE routers from a row-wise expert-proxy representation perspective and proposed Manifold Power Iteration (MPI). MPI is an efficient and theoretically grounded alternative to conventional router designs, and establishes a principled connection between router representations and expert parameters. It requires only lightweight iterative updates while maintaining scalability. Extensive experiments validates MPI across diverse architectures and training settings. We hope this work inspires future research on mathematically principled router design and advances the understanding of the representation geometry in MoEs.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.

[3] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa

Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.

[4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[5] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4599–4610, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.365. URL https://aclanthology.org/2021.naacl-main.365/.

[6] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.

[7] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2368–2378, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1246. URL https://aclanthology.org/N19-1246/.

[8] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1:211–218, 1936. URL https://api.semanticscholar.org/CorpusID:10163399.

[9] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems*, 5:288–304, 2023.

[10] GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng

Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering, 2026. URL https://arxiv.org/abs/2602.15763.

[11] Gene H. Golub and Charles F. Van Loan. *Matrix computations (3rd ed.)*. Johns Hopkins University Press, USA, 1996. ISBN 0801854148.

[12] Yuling Gu, Oyvind Tafjord, Bailey Kuehl, Dany Haddad, Jesse Dodge, and Hannaneh Hajishirzi. Olmes: A standard for language model evaluations, 2025. URL https://arxiv.org/abs/2406.08446.

[13] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions, 2010. URL https://arxiv.org/abs/0909.4061.

[14] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021. URL https://arxiv.org/abs/2009.03300.

[15] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams, 2020. URL https://arxiv.org/abs/2009.13081.

[16] Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

[17] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay

and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147/.

[18] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

[19] Jon M. Laurent, Joseph D. Janizek, Michael Ruzo, Michaela M. Hinks, Michael J. Hammerling, Siddharth Narayanan, Manvitha Ponnapati, Andrew D. White, and Samuel G. Rodriques. Lab-bench: Measuring capabilities of language models for biology research, 2024. URL https://arxiv.org/abs/2407.10362.

[20] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, Sanket Purandare, Gokul Nadathur, and Stratos Idreos. Torchtitan: One-stop pytorch native solution for production ready LLM pretraining. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=SFN6Wm7YBI.

[21] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

[22] Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. Fineweb-edu: the finest collection of educational content, 2024. URL https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu.

[23] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. Olmoe: Open mixture-of-experts language models, 2025. URL https://arxiv.org/abs/2409.02060.

[24] Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander

Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg, Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. Olmo 3, 2025. URL https://arxiv.org/abs/2512.13961.

[25] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

[26] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In Gerardo Flores, George H Chen, Tom Pollard, Joyce C Ho, and Tristan Naumann, editors, *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR, 07–08 Apr 2022. URL https://proceedings.mlr.press/v174/pal22a.html.

[27] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1144. URL https://aclanthology.org/P16-1144/.

[28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.

[29] Thomas Pethick, Wanyun Xie, Kimon Antonakopoulos, Zhenyu Zhu, Antonio Silveti-Falls, and Volkan Cevher. Training deep learning models with norm-constrained lmos. In *Forty-second International Conference on Machine Learning*.

[30] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020. URL https://arxiv.org/abs/1910.02054.

[31] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1264. URL https://aclanthology.org/D16-1264/.

[32] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions, 2017. URL https://arxiv.org/abs/1710.05941.

[33] Siva Reddy, Danqi Chen, and Christopher D. Manning. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019. doi: 10.1162/tacl_a_00266. URL https://aclanthology.org/Q19-1016/.

[34] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.

[35] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. Socialiqa: Commonsense reasoning about social interactions, 2019. URL https://arxiv.org/abs/1904.09728.

[36] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[37] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, , and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.

[38] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1421. URL https://aclanthology.org/N19-1421/.

[39] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Chenxiao Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Qizheng Gu, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Yangyang Hu,

Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yang Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Haoyu Lu, Lijun Lu, Yashuo Luo, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Zeyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Lin Sui, Xinjie Sun, Flood Sung, Yunpeng Tai, Heyi Tang, Jiawen Tao, Qifeng Teng, Chaoran Tian, Chensi Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Si Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Haoning Wu, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Jing Xu, Jing Xu, Junjie Yan, Yuzi Yan, Hao Yang, Xiaofei Yang, Yi Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Siyu Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Shaojie Zheng, Longguang Zhong, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2026. URL https://arxiv.org/abs/2507.20534.

[40] David Wadden, Kejian Shi, Jacob Morrison, Alan Li, Aakanksha Naik, Shruti Singh, Nitzan Barzilay, Kyle Lo, Tom Hope, Luca Soldaini, Shannon Zejiang Shen, Doug Downey, Hannaneh Hajishirzi, and Arman Cohan. SciRIFF: A resource to enhance language model instruction-following over scientific literature. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 6072–6109, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.310. URL https://aclanthology.org/2025.emnlp-main.310/.

[41] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In Leon Derczynski, Wei Xu, Alan Ritter, and Tim Baldwin, editors, *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pages 94–106, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-4413. URL https://aclanthology.org/W17-4413/.

[42] Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic pretraining

optimizers and where to find them. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=2J51qUZ0iG.

[43] Tian Xie, Haoming Luo, Haoyu Tang, Yiwen Hu, Jason Klein Liu, Qingnan Ren, Yang Wang, Wayne Xin Zhao, Rui Yan, Bing Su, Chong Luo, and Baining Guo. Controlled llm training on spectral sphere, 2026. URL https://arxiv.org/abs/2601.08393.

[44] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472. URL https://aclanthology.org/P19-1472/.

[45] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. St-moe: Designing stable and transferable sparse expert models, 2022. URL https://arxiv.org/abs/2202.08906.

A Supplementary Derivations for Approximation in Equation 10

In what follows, we provide detailed derivation for weights update Δr of router $R'_{[i]}$ within Manifold Power Iteration. Formally, Δr is given as:

$$\Delta r_M = \frac{R'_{[i]}M}{\|R'_{[i]}M\|_2} - R'_{[i]},$$

where $\frac{R'_{[i]}M}{\|R'_{[i]}M\|_2}$ denotes the updated $R'_{[i]}$ via power iteration. We project $R'_{[i]}M$ onto the subspace spanned by $R'_{[i]}$ and its orthogonal complement:

$$\begin{aligned} R'_{[i]}M \quad &= \quad R'_{[i]}\left(R'_{[i]}MR'^{\top}_{[i]}\right) \\ &+ \underbrace{\left(R'_{[i]}M - R'_{[i]}\left(R'_{[i]}MR'^{\top}_{[i]}\right)\right)}_{\text{orthogonal to } R'_{[i]}}. \end{aligned}$$

As the power iteration proceeds, $R'_{[i]}$ asymptotically toward the dominant subspace, and the orthogonal component (the second term above) becomes negligible. Consequently, we can arrive at the following approximation:

$$\frac{R'_{[i]}M}{\|R'_{[i]}M\|_2} \approx R'_{[i]} + \frac{R'_{[i]}M - R'_{[i]}(R'_{[i]}MR'^{\top}_{[i]})}{R'_{[i]}MR'^{\top}_{[i]}},$$

through a simple rearrangement of terms, we obtain

$$\Delta r_M \approx \frac{1}{R'_{[i]}MR'^{\top}_{[i]}}\left(R'_{[i]}M - R'_{[i]}(R'_{[i]}MR'^{\top}_{[i]})\right).$$

which completes the derivation of Eq. 10.

B Details for Pretraining Experiments

B.1 Implementation Details

Table 7 summarizes the hyperparameters of the model architectures across experiments. To support large-scale experiments, we scale our 3B model to 11B by expanding the experts counts from 64 to 256, resulting in a sparse MoE model with 11B total parameters and 470M activated parameters.

Our training pipeline is built upon the TorchTitan framework [20]. For Transformer components, we adopt PyTorch's SDPA for attention [28], and MegaBlocks MLP for efficient MoE implementation [9]. In terms of model parallelism, we adopt Fully Sharded Data Parallel [30] across all pretraining experiments.

B.2 Optimizer Setup Details

Table 8 presents the hyperparameters used for pretraining the 1B MoE model with AdamW. This parameter set was identified through our hyperparameter search over key configurations. For pretraining 1B models with other optimizers, we simply align their update RMS with AdamW. This ensures fair convergence comparisons and avoid the cost for extensive hyperparameter search.

For Muon, this alignment equates to scaling the learning rate of Muon-optimized parameters by $0.2 \times \sqrt{\max(d_{in}, d_{out})}$ [39]. More details regarding our Muon implementation are available in the code.

For Hyperball Optimization, we fix the Frobenius norm of the weight matrix $W \in \mathbb{R}^{d_{in} \times d_{out}}$ at $\sqrt{d_{out}}$. To align the update RMS, this translates to a learning rate scaler of $0.2 \times \sqrt{d_{in}}$. We substitute the d_{in} value from a 1B model into this formula, using the resulting value as a scale-invariant constant across all model scales.

C Evaluation Setup

We perform all downstream task evaluations using OLMES [12]. During pretraining, we evaluate the model on 9 core tasks—ARC-Easy [3], ARC-Challenge [3], MMLU [14], CommonsenseQA [38], SocialIQA [35], HellaSwag [44], WinoGrande [34], PIQA [2], and SciQ [41]—to quickly assess the fundamental capabilities of these checkpoints. Unless otherwise specified, we follow [24] and evaluate on a benchmark consisting of 25 multiple-choice tasks; a complete list of these tasks is provided in Table 9. To save space, we do not report the detailed scores for these 25 tasks unless explicitly specified.

Configuration	1 B	3 B	11 B
Dimension	1024	1536	1536
Layers	8	12	12
Attention Heads	8	16	16
Key-value Heads	8	8	8
RoPE Theta (θ)	500,000	500,000	500,000
FFN Dimension	512	768	768
Activated Experts	8	8	8
Routed Experts	64	64	256
Sequence Length	2048	4096	4096
Vocab Size	128,256	128,256	128,256
# Dense Params	296 M	479 M	479 M
# Sparse Params	806 M	2.72 B	10.88 B
# Activated Params	397 M	823 M	823 M
# Total Params	1.10 B	3.20 B	11.36 B

TABLE 7 Hyperparameters of model architectures.

Configuration	1B
Optimizer	AdamW
(β_1, β_2)	$(0.9, 0.95)$
Peak LR	1.0E-03
Minimum LR	0
LR scheduler	cosine
Warmup steps	1000
Weight decay	0.1
Gradient clip	1.0
LBL weight	0.01

TABLE 8 Pretraining hyperparameters (1B AdamW).

	AdamW		AdamH		Muon		MuonH	
	MoE	*w.* MPI	MoE	*w.* MPI	MoE	*w.* MPI	MoE	*w.* MPI
Arc-Easy [3]	59.64	64.05	64.09	66.88	60.65	62.16	61.82	65.36
Arc-Challenge [3]	32.51	36.26	35.15	37.37	34.90	35.75	35.24	33.62
MMLU-Stem [14]	28.13	27.80	28.19	28.66	28.26	27.60	27.87	28.33
MMLU-Humanities [14]	34.38	34.68	35.03	35.55	33.77	34.38	33.96	34.51
MMLU-Social Science [14]	29.12	29.12	28.20	29.08	28.48	29.50	29.25	29.07
MMLU-Other [14]	33.41	35.84	35.47	37.05	35.35	35.84	33.47	36.64
CSQA [38]	38.25	48.73	38.00	48.08	44.14	45.37	42.83	46.76
HellaSwag [44]	47.18	47.68	48.52	48.35	46.52	48.16	46.31	45.24
WinoGrande [34]	52.25	51.93	51.70	51.93	51.78	52.01	52.25	51.93
SocialIQA [35]	41.91	44.58	42.02	43.86	42.99	44.11	41.91	43.86
PiQA [2]	69.04	68.82	69.04	69.91	68.61	69.64	67.74	67.90
CoQA [33]	47.09	52.39	47.00	37.84	54.37	53.61	51.27	54.00
DROP [7]	30.79	27.71	26.83	29.03	28.53	28.34	28.72	28.15
Jeopardy	46.92	43.82	50.02	54.51	49.02	49.64	52.98	53.36
NaturalQs [18]	24.31	25.76	23.71	25.57	25.31	23.51	24.21	26.11
SQuAD [31]	49.91	48.96	50.47	51.04	49.86	48.63	53.65	52.70
SciQ [41]	70.90	76.70	74.50	76.80	73.20	73.60	72.30	72.60
QASPER [5]	67.40	68.02	58.93	67.40	62.07	68.03	61.44	67.40
DBQA [19]	27.12	23.84	26.15	25.38	24.81	26.54	23.85	26.35
ProtocalQA [19]	25.00	27.78	26.85	27.78	30.56	27.78	26.85	29.63
Lambada [27]	39.34	39.39	41.57	40.15	39.76	39.24	38.95	40.99
MedMCQA [26]	27.16	28.26	26.99	28.66	27.32	28.16	28.42	28.14
MedQA [15]	23.17	23.80	24.04	24.90	24.90	23.57	21.84	23.96
SciRIFF [40]	72.44	72.44	71.93	72.82	71.11	71.87	72.44	72.00
Basic Skills [24]	39.23	40.73	40.27	39.73	39.03	39.54	39.90	40.78
Avg RC Acc.	42.26	43.93	42.59	43.93	43.01	43.55	42.78	43.98

TABLE 9 Task-specific performance comparisons for 1B MoE with different optimizers.

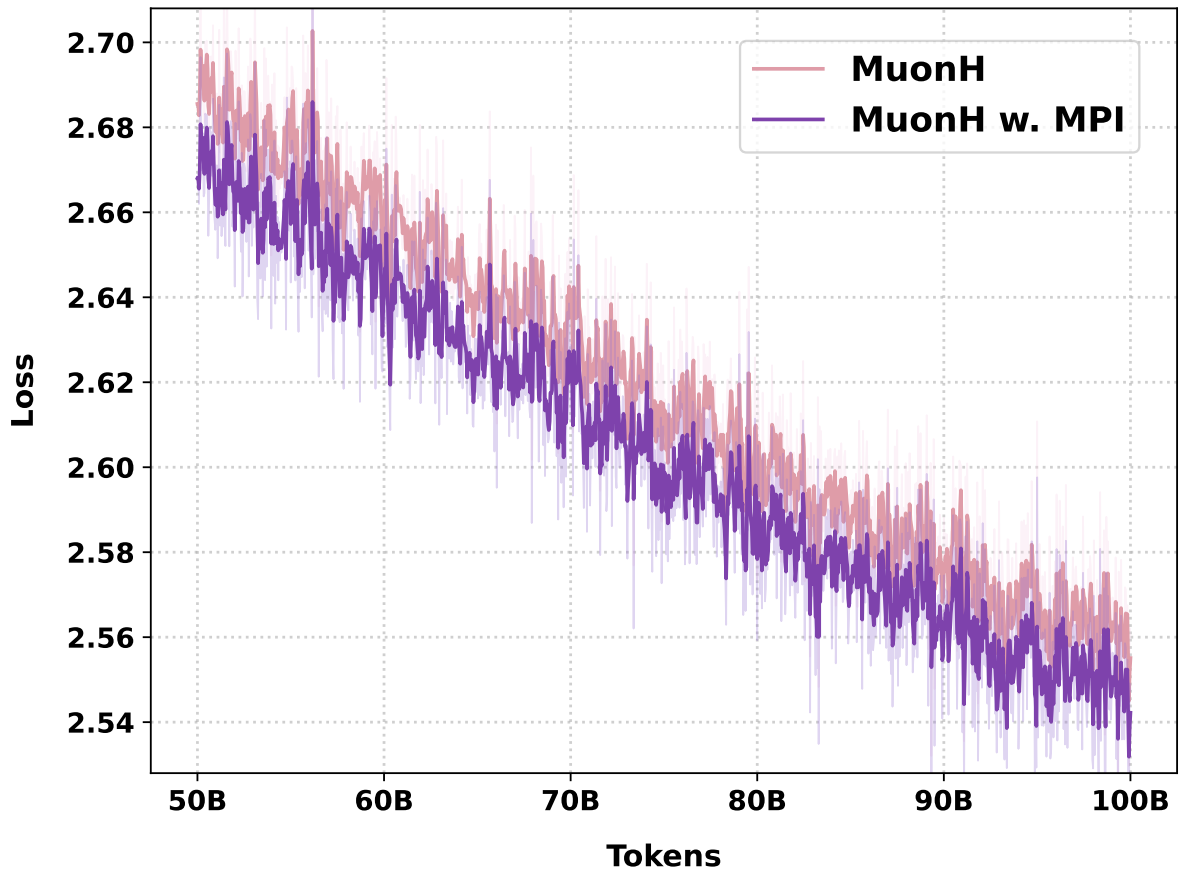
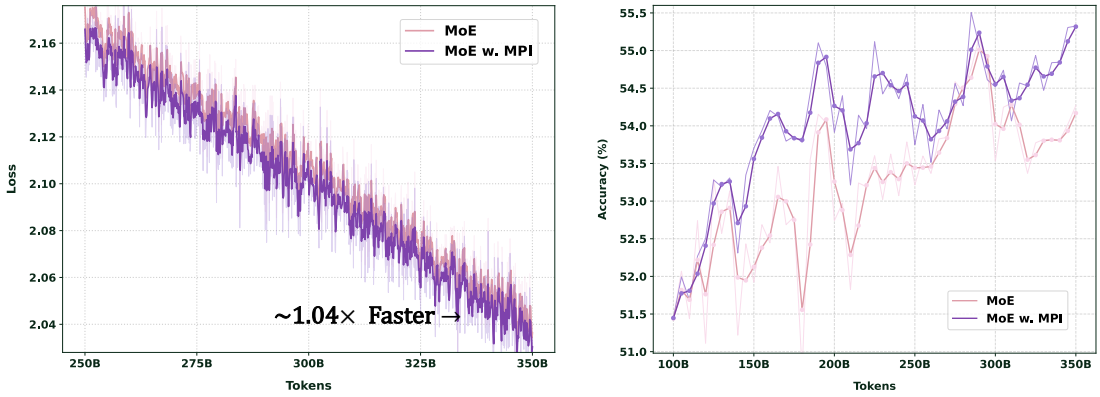


FIGURE 2 Convergence comparisons for MoE with MPI, exemplified by MuonH-1B. Our router design achieves a 0.013 reduction in pretraining loss. Similar observations for other optimizers are provided in the Appendix.

	AdamW	AdamH	Muon	MuonH
MoE	42.26	42.59	43.01	42.78
− MPI	43.56	43.93	43.55	43.98

TABLE 1 Downstream performance (average accuracy across 25 benchmarks). MPI consistently improves downstream performance across different optimizers. Detailed task-specific results are provided in Table 9. In the remainder of this paper, unless otherwise specified, we only report the average results across the 25 tasks.



(a) Pretraining Loss Comparison (b) Average Downstream Task Accuracy

FIGURE 3 Convergence and Downstream Performance Comparison. Manifold Power Iteration facilitates faster convergence and superior downstream task performance throughout the entire course of 11B MoE pretraining.

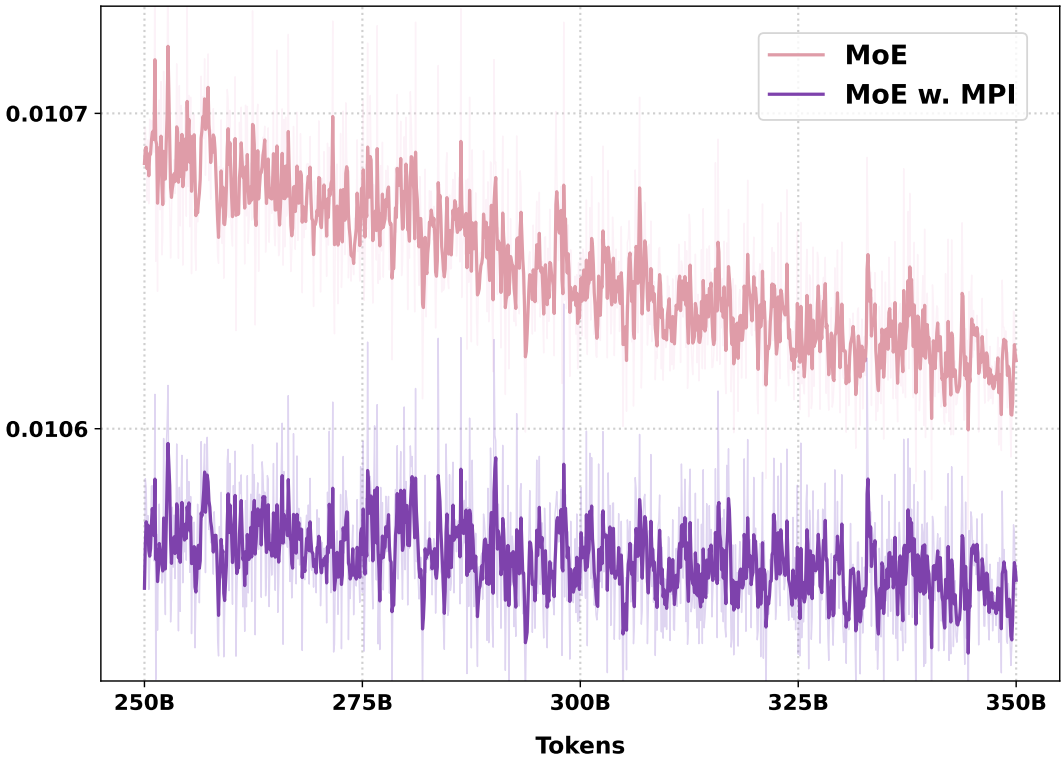


FIGURE 4 Load balancing loss for 3B MoE with MPI.

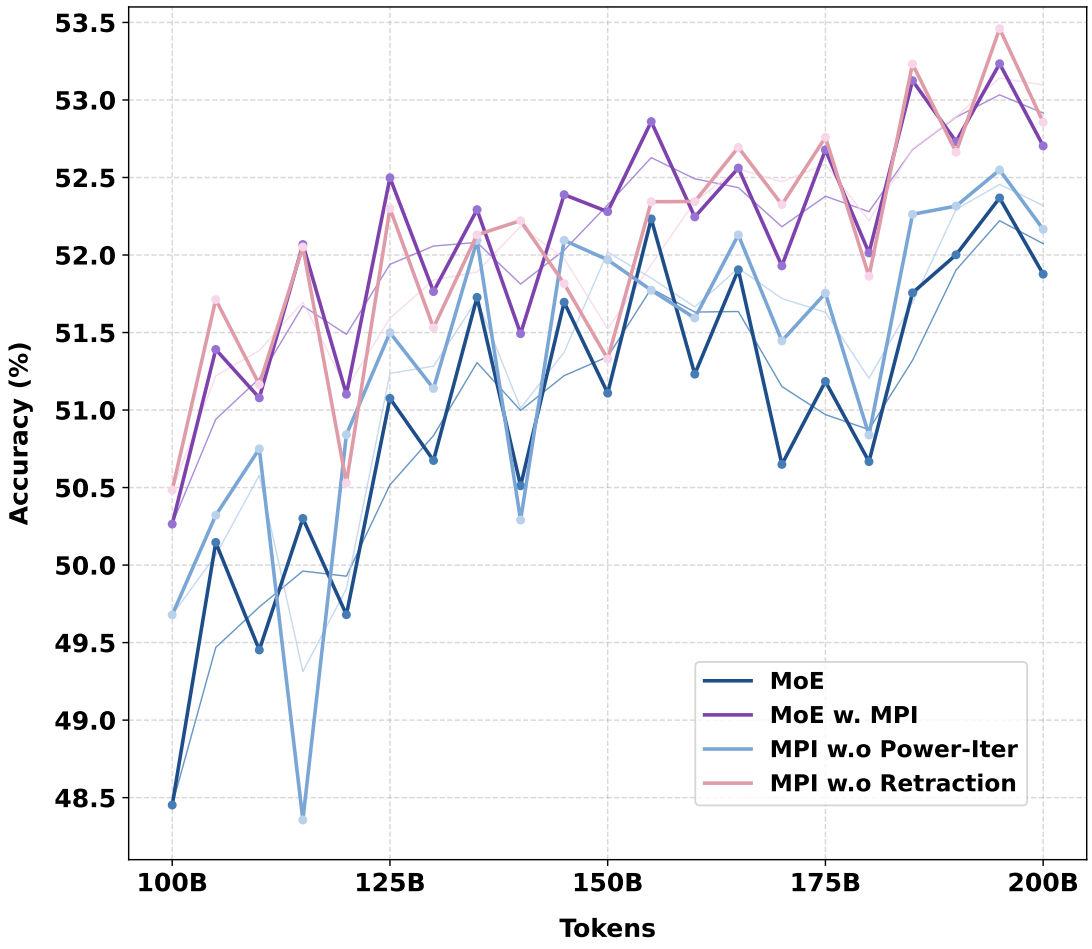


FIGURE 5 Ablation studies for the key design choices: (1) Power Iteration and (2) Router Retraction. We observe pretraining collapses without Router Retraction when using AdamW and Muon, showcasing that Router Retraction is critical for maintaining training stability, especially for optimizers that lack weight constraints.

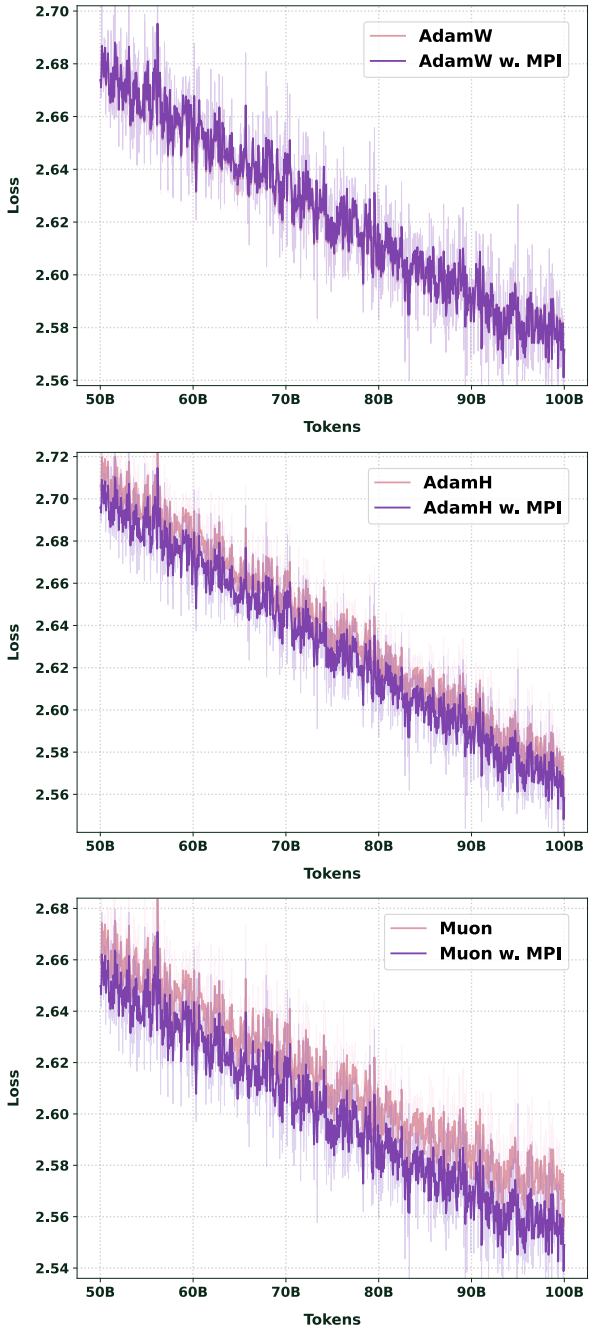


FIGURE 6 Pre-training loss comparison for a 1B MoE model across optimizers (AdamW, AdamH, Muon). MoE with MPI achieves a convergence advantages over all alternative setups.